LAB MANUAL

Web Application Development using Ruby on Rails

1. Aim

To develop a simple web application using the Ruby on Rails framework and demonstrate MVC architecture, routing, database interaction, forms, and basic CRUD operations.

2. Learning Objectives

- Understand the basic architecture and working of Ruby on Rails.
- Create and configure a Rails web application.
- Implement the Model–View–Controller (MVC) architecture.
- Create a database model and perform database migration.
- Implement Create, Read, Update, and Delete (CRUD) operations.
- Configure routes, controllers, views, and forms in a Rails application.

3. Software / Hardware Requirements

Requirement	Specification
Operating System	Windows / Linux / macOS
Programming Language	Ruby
Framework	Ruby on Rails
Database	SQLite3
Code Editor	VS Code / Sublime Text / Any IDE
Web Browser	Chrome / Firefox / Edge
Terminal	Command Prompt / PowerShell / Terminal

4. Theory

Ruby on Rails (Rails) is an open-source web application framework written in Ruby. It is designed for developing database-backed web applications efficiently. Rails follows the Model–View–Controller (MVC) architecture and the principle of Convention over Configuration.

MVC Components:

Component	Function
Model	Manages application data, business logic, and database interaction.
View	Presents the user interface and displays data to the user.
Controller	Handles HTTP requests and coordinates between the model and view.

5. Problem Statement

Develop a Student Management System using Ruby on Rails. The application should allow the user to add, view, edit, and delete student records containing the student's name, email, and course.

6. Procedure

Step 1: Verify Ruby and Rails Installation

Open a terminal and check the installed Ruby version:

```
ruby --version

rails --version
```

If Rails is not installed, install it using:

```
gem install rails
```

Step 2: Create a New Rails Application

Create a project named StudentApp and move into its directory:

```
rails new StudentApp

cd StudentApp
```

Step 3: Create the Student Model

Generate the Student model with name, email, and course attributes:

```
rails generate model Student name:string email:string course:string

rails db:migrate
```

The migration creates a students table with id, name, email, course, created_at, and updated_at fields.

Step 4: Generate the Controller

Generate the Students controller:

```
rails generate controller Students index
```

Replace the contents of app/controllers/students_controller.rb with:

```
class StudentsController < ApplicationController
  def index
    @students = Student.all
  end

  def new
    @student = Student.new
  end

  def create
    @student = Student.new(student_params)
    if @student.save
      redirect_to students_path
    else
      render :new
    end
  end

  def edit
    @student = Student.find(params[:id])
```

```
  end

  def update
    @student = Student.find(params[:id])
    if @student.update(student_params)
      redirect_to students_path
    else
      render :edit
    end
  end

  def destroy
    @student = Student.find(params[:id])
    @student.destroy
    redirect_to students_path
  end

  private

  def student_params
    params.require(:student).permit(:name, :email, :course)
  end
end
```

Step 5: Configure Routes

Open config/routes.rb and add:

```
Rails.application.routes.draw do
  resources :students
  root "students#index"
end
```

Step 6: Create the Index View

Create/open app/views/students/index.html.erb:

```
<h1>Student Management System</h1>

<%= link_to "Add New Student", new_student_path %>
<br><br>

<table border="1" cellpadding="10">
  <tr>
    <th>Name</th>
    <th>Email</th>
    <th>Course</th>
    <th>Actions</th>
  </tr>

  <% @students.each do |student| %>
    <tr>
      <td><%= student.name %></td>
      <td><%= student.email %></td>
      <td><%= student.course %></td>
      <td>
        <%= link_to "Edit", edit_student_path(student) %> |
        <%= link_to "Delete", student_path(student),
            data: { turbo_method: :delete,
                    turbo_confirm: "Are you sure?" } %>
```

```
      </td>
    </tr>
  <% end %>
</table>
```

Step 7: Create the New Student Form

Create app/views/students/new.html.erb:

```
<h1>Add New Student</h1>

<%= form_with model: @student do |form| %>
  <p>
    <%= form.label :name %><br>
    <%= form.text_field :name %>
  </p>
  <p>
    <%= form.label :email %><br>
    <%= form.email_field :email %>
  </p>
  <p>
    <%= form.label :course %><br>
    <%= form.text_field :course %>
  </p>
  <p>
    <%= form.submit "Save Student" %>
  </p>
<% end %>

<%= link_to "Back", students_path %>
```

Step 8: Create the Edit View

Create app/views/students/edit.html.erb:

```
<h1>Edit Student</h1>

<%= form_with model: @student do |form| %>
  <p>
    <%= form.label :name %><br>
    <%= form.text_field :name %>
  </p>
  <p>
    <%= form.label :email %><br>
    <%= form.email_field :email %>
  </p>
  <p>
    <%= form.label :course %><br>
    <%= form.text_field :course %>
  </p>
  <p>
    <%= form.submit "Update Student" %>
  </p>
<% end %>

<%= link_to "Back", students_path %>
```

Step 9: Run the Application

Start the Rails server:

```
rails server
```

Open the following address in a browser:

```
http://localhost:3000
```

7. Expected Output

The home page displays the Student Management System with an option to add, edit, and delete student records.

Sample output:

Name	Email	Course	Actions
Rahul	rahul@gmail.com	AI & DS	Edit | Delete
Priya	priya@gmail.com	Computer Engineering	Edit | Delete

8. CRUD Operations

Operation	Rails Action	Purpose
Create	create	Add a new student record
Read	index / show	Display student records
Update	update	Modify student details
Delete	destroy	Remove a student record

9. Result

The Student Management System was successfully developed using Ruby on Rails. The experiment demonstrated MVC architecture, routing, database migration, controllers, views, forms, and CRUD operations.

10. Viva Voce Questions

1. What is Ruby on Rails?

2. What is MVC architecture?

3. What is the role of a Rails model?

4. What is the purpose of a controller?